Stingray: Fast Concurrent Transactions Without Consensus

Srivatsan Sridhar
Stanford University

Alberto Sonnino
Mysten Labs
University College London

Lefteris Kokoris-Kogias
Mysten Labs

Abstract

Blockchain systems face a fundamental tension between performance and security, particularly in processing transactions concurrently. While consensus-less protocols offer promising latency benefits by concurrently processing transactions accessing disjoint parts of the state, they struggle with three critical challenges: (1) the inability to process operations on shared state concurrently, (2) limited support for multi-party transactions, due to liveness concerns, and (3) slow recovery from contention, often requiring day-long epoch changes. These limitations force existing systems to make conservative design choices, significantly constraining their performance and practical utility.

We present Stingray, a novel blockchain architecture that fundamentally advances the state-of-the-art in Byzantine fault-tolerant distributed systems. Our system fundamentally reimagines how concurrent transactions can be processed securely without consensus by separating the handling of true contention (non-commutative operations) from simple concurrency (operations that can be parallelized). Stingray introduces a hybrid approach that maximizes fast-path execution while providing formal safety and liveness guarantees in asynchronous networks, even under Byzantine failures. Additionally, Stingray provides a way to recover from loss of liveness due to contention instantly through its novel FastUnlock protocol. We prove Stingray's security in an asynchronous network with Byzantine faults and demonstrate on a global testbed that Stingray achieves 10,000 times the throughput of prior systems for commutative workloads.

CCS Concepts

• **Security and privacy** → **Distributed systems security**; • **Computer systems organization** → **Dependable and fault-tolerant systems and networks**; • **Theory of computation** → *Distributed computing models*; • **Software and its engineering** → *Concurrency control*.

Keywords

Consensus-less transactions, Byzantine fault tolerance, Blockchain, Concurrent execution, Commutativity

ACM Reference Format:
Srivatsan Sridhar, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2026. Stingray: Fast Concurrent Transactions Without Consensus. In *Proceedings of the 33rd ACM Conference on Computer and Communications Security (CCS '26), November 15–19, 2026, The Hague, Netherlands.* ACM, New York, NY, USA, 18 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '26, The Hague, Netherlands
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/11
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Sub-second confirmation latency is increasingly a baseline requirement for blockchains. However, most blockchains still use BFT consensus (Fig. 1a) to totally order *all* transactions, imposing an avoidable latency penalty on *parallelizable* (independent or near-commutative) transactions.

In the last five years, theoretical breakthroughs have shown that consensus is unnecessary for a large class of distributed transactions [24], but despite this compelling promise, only a few blockchains have adopted this consensus-less *fast-path* (Fig. 1b) design [12, 33, 37]. This is because the fast-path is currently triggered only for specific transaction types while the majority of the load is assigned to consensus far more often than ideal. Three key factors hinder the mass adoption of the fast-path design paradigm:

(1) Intra-object serialization: Transactions on the same object or key that effectively commute (e.g., small debits well below an account's balance, interleaved credits and debits) are forced through a single sequential chain (Fig. 1c), wasting parallelism and limiting throughput. An important transaction type that suffers from this limitation is payments (including gas payments).
(2) Single-owner scope: Fast-path execution is limited to assets explicitly owned by a single user [9, 10, 19, 30, 33, 37, 42]; any atomic multi-owner operation (swap, joint spend, programmable authorization) resorts to consensus to avoid the risk of locking due to conflicting transactions sent by different owners. Such transactions get no latency benefits.
(3) Coarse recovery: Upon conflicting or equivocal transactions (multi-owner transactions, buggy clients, adversarial behavior), systems like Sui [30] require a once-in-a-day reset to reconcile state, stalling progress and exploding tail latency.

Collectively these factors cap throughput, raise confirmation latency, and limit the benefits of adopting the fast path. As a result, the fast path is not yet widely adopted by blockchain systems.

Our answer to the current status-quo is Stingray, a consensus-less blockchain execution architecture that resolves all the above tensions, unleashing the true power of consensus-less validation. Specifically, we solve the above three challenges by:

(1) Introducing a BFT bounded counter that extracts parallelism from commutative and **nearly commutative transactions** (Fig. 1d).
(2) Providing a fast recovery mechanism from conflicting or equivocal transactions using a **FastUnlock protocol**.
(3) Allowing **multi-owner transactions** that carry a small risk of contention to use the fast path, and recover using FastUnlock if needed.

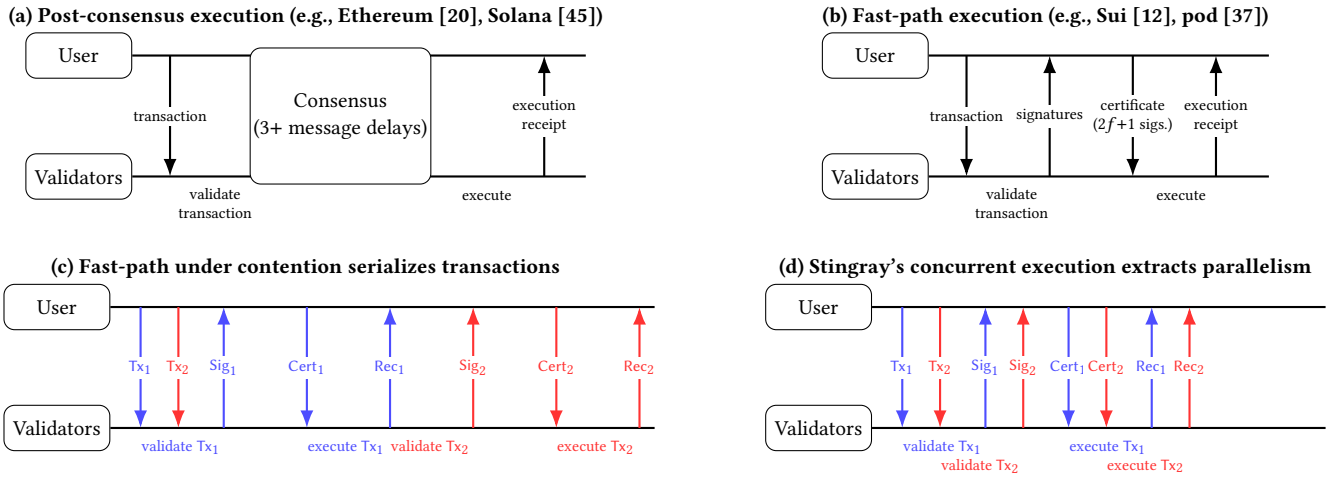

Figure 1: Comparison between post-consensus execution, fast-path execution, and concurrent execution (Stingray's bounded counters). While consensus requires at least three communication steps, fast-path execution occurs after two, thus decreasing latency for transactions that do not access shared state. However, fast-path execution still serializes transactions that access the same object, wasting parallelism and limiting throughput. Stingray's concurrent execution allows commutative and nearly commutative transactions on the same object to be executed parallelly, enhancing throughput.

The result is a general blockchain system that supports a broad set of applications that circumvent consensus, relegating the use of it for rare, surgical contention resolution.

Nearly Commutative Transactions. CRDTs (Conflict-free Replicated Data Types) [41] allow replicas in a distributed system to execute commutative transactions in parallel and without consensus, but they do not capture anything beyond purely commutative operations. Payments, however, require a non-commutative comparison with zero to ensure that the balance is not overdrawn. Yet payments can be considered *nearly commutative* when the amount is well below the account's balance and can potentially be executed in parallel (as in Fig. 1d). Escrow and allocation bounded counters [1, 36, 44] address this problem in the crash-fault setting but break under Byzantine collusion: a malicious minority can approve transactions spending more than the available balance. The open problem is to support *nearly commutative* addition and subtraction operations subject to a non-negativity invariant, in an asynchronous BFT setting, without reintroducing global ordering.

We solve this problem by introducing the first Byzantine fault-tolerant bounded counter construction. The key insight to designing a BFT bounded counter is to limit the amount that can be concurrently spent from a bounded counter. Unlike prior work in the CFT setting where the entire balance could be spent concurrently, in Stingray, we only allow about half of the balance to be spent concurrently before a synchronization step must occur. That is, each validator is allowed a *budget* that is half of the balance and can only sign transactions spending up to that budget. When this is combined with the assumption that at most 1/3 of the validators are malicious, we can ensure that the set of executed transactions never exceeds the available balance. However, the budget also limits the number of transactions that can be executed concurrently. To make sure that the counter can be used completely, we enable the owner of the counter to issue an asynchronous version update during which validators reset their budget based on already certified transactions.

FastUnlock Protocol. The second key insight of Stingray is its approach to resolving contention. When an object experiences contention, such as conflicting transactions, validators perform an atomic snapshot [16, 35] of the object's current state. This snapshot collects the views of $2f+1$ validators and maintains in the execution result any transaction that could have been finalized in the views of any client. If a transaction is in progress but not yet widely executed, it may not appear consistently executed across validators. To prevent an inconsistent state from compromising the liveness of the system, the snapshot is performed with the assistance of the underlying consensus engine [4, 43], forcing a uniform view of the state across the system and safely resolving contention.

Multi-owner Transactions. The existence of FastUnlock for quick contention resolution extends fast-path eligibility to multi-owner atomic transactions (swaps, joint spends, etc.) where contention is possible but rare. Such transactions run optimistically and invoke FastUnlock only under genuine contention, minimising latency and executing concurrently, without forfeiting liveness for long periods as in prior work [4, 12].

We implement Stingray atop Mysticeti-FPC [4], the core consensus-less fast path of Sui [30]. Our evaluation shows dramatic improvements: bounded counter-based single-object workloads jump from 2 tps to 20,000 tps with sub-second latency—over four orders of magnitude faster while other workloads retain baseline throughput.

Stingray can be integrated into *any blockchain* to broaden the range of transactions that benefit from a consensus-less fast path and to enable rapid recovery from contention. This makes the fast-path architecture significantly more appealing for developers and

applications. With Stingray, consensus-less execution becomes the default mode, while ordered consensus is only invoked as a brief, targeted response to contention.

Contributions. This paper makes the following contributions:

(1) It introduces the first protocol that enables concurrent additions and subtractions under a non-negativity invariant.
(2) It presents the first safe consensus-less execution for multi-owner transactions with complex authorization.
(3) It introduces the first low-latency recovery mechanism that resolves contention within a consensus round.
(4) It formally proves the safety and liveness of Stingray in the asynchronous BFT model.
(5) It provides an open-source implementation demonstrating a 10,000× throughput improvement for bounded counter-based workloads and no penalty for parallel workloads.

2 Model

We introduce the transaction model, threat model, and security properties for Stingray. The Stingray protocol is run by $n = 3f + 1$ validators where an adversary can corrupt up to f validators.

2.1 Data Structures

The Stingray state consists of a set of objects. Transactions modify the state by consuming one or more input objects and creating one or more output objects.

Object Types. Objects are categorized based on their access control.

- *Owned objects* have an owner field that determines its access control. The owner is an address representing a public key. A transaction may modify the object only if the transaction is signed by that key (which can also be a multi-signature). An example is a coin object (similar to a Bitcoin UTXO) that can be transferred from one owner to another.
- *Shared objects* do not specify an owner. They can instead be included in any transaction and do not require any authorization. Instead, their authorization logic is enforced by a smart contract. Shared objects are used to represent the state of smart contracts with which multiple users interact.
- *Bounded counters* are a special case of owned objects. They have an owner and encapsulate a numerical value. Transactions decreasing the value must be signed by the owner while transactions increasing it can be signed by anyone subject to a smart contract's logic. A canonical example is the cryptocurrency account of a user where the value is the user's balance.

Owned and shared objects are standard in object-based blockchains [12, 33, 37]. Stingray introduces a third category, bounded counters (formally defined in Sec. 3.1), which enables concurrent access to shared numerical state.

Transactions. A transaction is a signed command that specifies one or more input objects, a version number per object, and optionally a set of parameters. Executing the transaction consumes the input objects and constructs a set of output objects—which may be the input objects at a higher version number or new objects.

Certificates. A *certificate* (Cert) on a transaction contains the transaction and signatures from a quorum of at least $2f + 1$ validators with their identifiers. A certificate may not be unique, and the same logical certificate may be signed by different quorums of validators. However, two different valid certificates on the same transaction are treated as representing semantically the same certificate.

2.2 Transaction Execution Modes

Post-Consensus Execution. In general, the validity of a transaction and its effects depend on the transactions executed before it. Thus, most blockchains execute transactions after they are totally ordered through a consensus protocol (Fig. 1a). Even in fast-path systems [12, 37], all transactions except those eligible for the fast path are executed post-consensus.

Fast Path Execution. A transaction is said to be executed in the fast path if validators execute the transaction before it is totally ordered with respect to other transactions through a consensus protocol. Validators execute a transaction in the fast path upon seeing a certificate for that transaction (Fig. 1b). In Stingray and prior fast-path systems like Sui [12], a transaction is executed in the fast path if all its input objects are owned objects with the same owner. An example is a transfer of a coin object from one owner to another (i.e., a payment).

To ensure that all validators execute transactions in the same order, validators execute transactions in increasing order of the version numbers of their input objects. That is, validators execute a transaction that inputs an object at version v only after executing a transaction that inputs the same object at version $v - 1$. Thus, the object's owner already specifies the order of the transactions using the version numbers, thereby avoiding the need for consensus. Fig. 1c shows an example of fast-path execution of two transactions accessing the same owned object. Consensus requires at least three communication steps [4] for validators after receiving the user's transaction [34], whereas the fast path requires only two (signature and certificate in Fig. 1b). If the owner issues multiple transactions that input an owned object with the same version, there is no determined order, so these transactions must be executed post-consensus.

Concurrent Execution. While fast path execution provides lower latency for transactions that do not access shared state, it still has limited concurrency because validators must process transactions with the same object sequentially, as in Fig. 1c. In Stingray, bounded counters lift this limitation by enabling concurrent execution when possible. In Fig. 1d, Tx_2 is validated and executed without waiting for Tx_1 to be executed first. In general, different validators may even execute Tx_1 and Tx_2 in different orders, depending on which one they received earlier. Thus, Stingray allows concurrent execution when it does not result in inconsistent states, e.g., in commutative transactions and for bounded counters when the balance is far from zero. Concurrent execution of bounded counters is defined in Sec. 3.3.

Hybrid Consensus and Fast Path. Stingray and prior work [12] use a hybrid consensus and fast path mechanism, where some transactions are executed in the fast path while others are executed post-consensus. Fig. 2 shows how both execution modes work

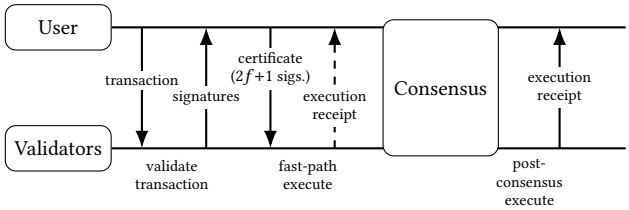

Figure 2: Hybrid fast-path and post-consensus execution of transactions in Stingray and in Sui.

together. Validators sign valid transactions, and a quorum of these signatures forms a certificate. If the transaction is eligible for fast-path execution, it can be executed and finalized immediately upon seeing the certificate; otherwise, the transactions are ordered via consensus and executed post-consensus.

2.3 Threat Model and Security Properties

The adversary is computationally bounded, ensuring that standard cryptographic properties such as the security of hash functions and digital signatures hold. Under this assumption, Stingray ensures *validity*, meaning that all transactions that are executed have valid authorization (Def. 1).

We consider a message-passing system with $n = 3f + 1$ validators running the Stingray protocol. An adversary can adaptively corrupt up to f validators, referred to as *Byzantine*, who may deviate arbitrarily from the protocol. The remaining validators, called *honest*, follow the protocol. The communication network is asynchronous and messages can be delayed arbitrarily. Given these conditions, Stingray is *safe*, that is, the union (merge) of all transactions executed by honest validators does not lead to invalid state transitions (Def. 3). This safety property subsumes the classic safety of blockchains since any fork in the state, if merged, would violate validity predicates such as conservation of value (e.g., in case of double-spends). At the same time, this definition captures concurrent execution.

Finally, assuming messages among honest validators are eventually delivered, Stingray is *live*, meaning honest validators eventually execute certifiably valid user transactions and update their state accordingly, and in the absence of new transactions, eventually converge to the same state (Def. 4). Here, liveness is required for transactions that can be executed without causing invalid state transitions, as certified by a quorum of validators (Def. 2). In contrast, requiring liveness for all transactions issued by honest users is too strong and unachievable, since the validity of their state transitions depends on the system's current state.

Definition 1 (Transaction Validity). *A transaction Tx is valid if all its input owned objects are owned by the transaction's signers.*

Definition 2 (Certificate Validity). *A transaction Tx has a valid certificate if:*

- *The transaction is valid*
- *It has a quorum certificate signed by at least $2f + 1$ validators*

Let $T_p(t)$ denote the set of transactions executed by validator p up to time t. The applications specify a set of predicates $\mathcal{P}$ over sequences of transactions. For example, a predicate may require that no user can spend more than their account balance or that no two transactions perform conflicting state updates.

Definition 3 (Safety Properties). *For any execution of Stingray with at most f Byzantine validators:*

- ***Validity:*** *For all p, t: all $Tx \in T_p(t)$ have valid certificates.*
- ***Global safety:*** *For all t, for all subsets H of honest validators, there exists a sequence T that contains $\bigcup_{p \in H} T_p(t)$ such that for all $P \in \mathcal{P}$: $P(T)$ is true.*

Definition 4 (Liveness Properties). *For any execution of Stingray with at most f Byzantine validators:*

- ***Progress:*** *Every transaction with a valid certificate whose input objects' owners are honest is eventually executed by all honest validators.*
- ***Eventual consistency:*** *For all honest validators p_1, p_2 and all time t, there exists a time $t' \geq t$ such that $T_{p_1}(t') \supseteq T_{p_2}(t)$.*

Alongside the above properties, our goal is to enable concurrent execution of as many transactions as possible. Stingray uses separate paths for processing commutative/bounded-counter transactions and for other fast-path transactions. We prove their security in Sec. B.1 and in Sec. B.3.

2.4 Motivating Applications of Stingray

We present three example applications of Stingray: one utilizing the bounded counter and two employing multi-owner transactions (whose potential liveness risks are mitigated by FastUnlock).

Concurrent Payments. The first application, concurrent debit or credit of an account, is one of the most common in existing blockchain systems. Not only is this useful for payments, but it is also useful for gas debits, which are required for every transaction. However, it presents significant challenges for parallel execution if not carefully designed. Incoming transactions (credits) only increase the account's balance, so they are commutative. Since they do not cause contention, they can easily be processed concurrently (even though prior blockchains do not do so). Concurrent debits, on the other hand, are more complex due to the need for zero-balance checks. These operations are non-commutative, hence they cause true contention that hinders concurrency.

Concurrent transactions may be achieved in UTXO-style blockchains by dividing one's account into smaller UTXOs. However, keeping track of these smaller UTXOs is cumbersome, and using the same UTXO twice is actually a double-spending attack. In account- or object-based blockchains, one can similarly split their account into smaller accounts and concurrently access them, but it has the same challenges as with UTXOs. In existing systems, user errors that cause contention can result in losing access until the next epoch boundary; often 24 hours. Stingray resolves this issue without introducing such complications by using a mostly-commutative bounded counter, allowing half of the budget to be spent concurrently before requiring a sequential rebalancing transaction.

Atomic swaps. Atomic swaps enable two parties to exchange digital assets without relying on a trusted intermediary. While consensus-based blockchains achieve this through smart contracts,

consensus-less environments face the risk of deadlock due to Byzantine users issuing concurrent transactions. Such scenarios can effectively lock both parties' assets. In existing fast-path systems, atomic swaps therefore require falling back to consensus, negating the latency benefits.

However, the risk of liveness loss on the fast path only occurs when an active attacker deliberately creates contention. Stingray provides an effective mitigation of this risk with the FastUnlock protocol. This safety net allows Stingray to support multi-owner transactions in the fast path, allowing fast path atomic swaps and other multi-party smart contracts, enhancing the programmability of consensus-less transactions. Although not a real application, the same risk runs for users who inadvertently equivocate on their objects, leading to unexpected deadlocks and a poor user experience. Here too, FastUnlock helps restore liveness quickly, lowering barriers to securely using an ultra-low latency blockchain.

Regulated stablecoins. Regulated stablecoins [18] require the issuer to be able to block an account for regulatory reasons, besides its owner spending from it. This has eluded consensus-less systems since sequencing these potentially conflicting operations requires consensus. Yet, the ability to block objects is nearly never exercised, creating no practical contention. Stingray's multi-owner transactions allow for complex access control and enable such transactions in the fast path where at any point the regulator can act as owner and force-send the stablecoin to a quarantined wallet.

2.5 Challenges

Stingray defines new object types that allow for higher concurrency. A Byzantine fault-tolerant CRDT [25] is sufficient for purely commutative data structures, but still cannot support transactions for blockchains because of gas payments that require a comparison of the user's balance with zero, a non-commutative operation.

To support concurrent payments (including gas payments), we define the *bounded counter*, which runs in the fast path. The design of this bounded counter creates our first challenge (**Challenge 1**): implementing nearly commutative objects in BFT settings. Prior work in the crash-fault tolerance (CFT) setting [1] simply splits the available budget among replicas and allows each replica to approve transactions spending up to its share of the budget. This doesn't work in the BFT setting since a single malicious replica could sign infinite transactions. As a result, we need to rely on quorums to distribute the budget collectively. Since there is an exponential number of potential quorums, we reduce this to giving a sufficient budget to each validator to spend concurrently but not enough to allow a minority of malicious validators to overspend. This tension results in our construction spending half of the budget concurrently when not under attack. We then show how we can reset the budget through a consistent read, allowing for the full amount to be concurrently spent with only $O(\log B)$ points of synchronization, where B is the initial balance.

The second challenge we take on is that existing consensus-less blockchains require transactions to operate on state owned by a single user, due to the fear of concurrency. Stingray removes the need for restricting concurrency in consensus-less transactions and instead identifies contention as the true culprit of correctness violations. Thus, Stingray enhances programmability by defining multi-owner transactions (e.g., an asset swap) that are processed in the fast path. This approach is powerful, and promises reduced latency for such operations that currently require consensus. But this creates our second challenge (**Challenge 2**): users may naturally submit conflicting transactions because contention is now possible. For example, two users may perform a swap and end up locking the objects due to bad timing. To address this challenge, Stingray uses a novel unlocking mechanism, called FastUnlock, that allows users to resolve conflicts quickly, enabling users to submit new transactions within the latency of a consensus protocol.

3 Concurrency through Commutativity

We present protocols that allow users to finalize a common category of transactions: updates to a *bounded counter*, without using consensus. This type of object is strictly harder to implement than CRDTs as it has a non-monotonic comparison with zero [32]. Our bounded counter protocol is a modification to the fast-path execution mode shown in Fig. 1b, while transactions on other owned objects continue to be processed as in Fig. 2.

3.1 The Bounded Counter Object

A bounded counter is an object that has a balance $\mathsf{Bal} \in \mathbb{R}$ as its state and supports transactions with a parameter $\delta \in \mathbb{R}$, allowing additions ($\delta > 0$) and subtractions ($\delta < 0$) on its balance, while maintaining the invariant $\mathsf{Bal} \geq 0$. Our goal is to execute transactions concurrently to the extent possible.

Definition 5 (Bounded counter). *A bounded counter object has an authorized user called the owner and an initial balance Bal_0, supports transactions Tx with value $Tx.\delta \in \mathbb{R}$ and has the following properties:*

1. ***Global safety**: as in Def. 3 with predicate $P(T) = (Bal_0 + \sum_{Tx \in T} Tx.\delta \geq 0)$.*
2. ***Progress**: If an honest owner sends a set of transactions T such that $Bal_0 + \sum_{Tx \in T} Tx.\delta \geq 0$, then all validators will eventually execute all transactions in T.*
3. ***Validity** and **eventual consistency**: as in Defs. 3 and 4.*

The progress property is a strengthening of Def. 4 wherein we specify the conditions under which transactions issued by honest owners get certified and eventually executed. We show a protocol in Sec. 3.3 and prove that it achieves the above properties in Sec. B.1.

The bounded counter is useful for a common use case of blockchains, cryptocurrencies, where an account's balance can be represented using a bounded counter. The bounded counter can be generalized to a bounded set in which elements can be added or removed, as long as the size of the set does not exceed a predefined bound. This may be useful, for example, to mint a predefined number of limited-edition non-fungible tokens (NFTs). More generally, the bounded counter can be used as a loop counter. For simplicity, we describe the most common case of a real-valued counter bounded below by 0 and consider payments from an account as the canonical use case.

3.2 Key Ideas for the Bounded Counter

Consider a user who owns an account containing Bal_0 units of money. As an example, suppose that each transaction the user

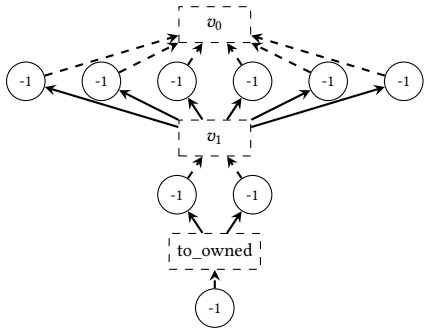

Figure 3: Version updates in the bounded counter. Circles represent unit decrement transactions and dashed boxes versions. The initial balance is $\mathsf{Bal}_0 = 9$, and $f = 1$. For version v_0, each validator has a budget of $\frac{f+1}{2f+1}\mathsf{Bal}_0 = 6$. After 6 transactions are certified, the user sends a version update (dashed box v_1) containing pointers to the 6 certified transactions. The remaining balance is $9 - 6 = 3$ and the validators update their budget to $\frac{f+1}{2f+1} * 3 = 2$. Finally, when the remaining balance is 1, the user converts the bounded counter to a standard owned object and spends the balance.

makes debits 1 unit from its account, i.e., $\mathsf{Tx}.\delta = -1$. In this case, safety requires that no honest validator execute more than Bal_0 transactions. Validators execute a transaction only if they see a certificate for the transaction, so it is sufficient to ensure that no more than Bal_0 transactions get certified.

Key Idea 1: Signing Budgets for Validators. Recall that Sui's fast path prevents the certification of two transactions on the same version of an object by ensuring that validators sign at most one transaction per version (Sec. 2.2). Extending this approach to allow concurrent transactions, we must ensure that each validator signs only a few transactions concurrently. We assign to each validator a *budget* $\mathsf{Bud} = \eta\mathsf{Bal}_0$, where $\eta = \frac{f+1}{2f+1}$, which is the maximum number of transactions the validator can sign. Since each certified transaction is signed by at least $2f+1$ validators, $f+1$ of whom are honest, for each certified transaction, at least $f+1$ is deducted from the total budget of all honest validators. Since the total budget of all honest validators starts at $(2f+1)\mathsf{Bud}$, the number of certified transactions can be at most $\frac{(2f+1)\mathsf{Bud}}{f+1} = \mathsf{Bal}_0$, even if the user and up to f validators are malicious.

Thus, the signing budgets ensure global safety. This idea alone does not satisfy liveness. If the f Byzantine validators abstain from signing transactions, each certified transaction requires signatures from $2f+1$ (all) *honest* validators, causing every honest validator to decrease their budget. So, at most $\mathsf{Bud} = \eta\mathsf{Bal}_0$ transactions will get certified, while liveness requires all of them to be certified eventually.

Key Idea 2: Version Updates. When the user realizes that validators may have exhausted their budget (because Bud transactions have already been certified), the user sends a *version update request* which includes pointers to all the previously certified transactions. Upon seeing a valid version update request, each validator updates their budget to η fraction of the balance remaining after executing the certified transactions. In the above example, this increases each validator's budget from 0 to $\eta(1-\eta)\mathsf{Bal}_0$. The validator also updates its local version to stop signing transactions with the previous version and start signing transactions with the new version (see Fig. 3 for an example).

Note that within a version, transactions get certified concurrently, while transactions across versions are certified sequentially. At each version, the user can spend up to η fraction of the remaining balance, until finally, when the remaining balance is small enough, the user can convert the bounded counter to a standard owned object and spend the remaining amount in one last transaction (shown in Fig. 3). Thus, with an initial balance of Bal_0 and each transaction spending 1 unit, the user requires only $\log(\mathsf{Bal}_0)$ version update requests, and therefore $O(\log(\mathsf{Bal}_0))$ latency to spend the entire balance. In contrast, this would require $O(\mathsf{Bal}_0)$ latency on Sui's fast path or any other consensus.

3.3 Bounded Counter Design

Alg. 1 shows the algorithm run by validators for the bounded counter. For simplicity, we specify the algorithm for when transactions have only one bounded counter object as input, although transactions with multiple bounded counters and other owned objects can be processed in the fast path.

Processing Transactions. For each bounded counter object, the state maintained by each validator includes the current version Version, the current budget Bud, and the set of transactions SignedTxs the validator has signed (lines 4 to 6). Upon receiving a new valid transaction, the validator checks that the transaction has the same version as the validator's current version (line 12). If the transaction's value $\mathsf{Tx}.\delta$ is within budget (line 13), the validator deducts its budget if required, marks the transaction as signed, and sends the signature to the user. Addition transactions ($\mathsf{Tx}.\delta > 0$) will always be signed, without decreasing the budget, while subtraction transactions ($\mathsf{Tx}.\delta < 0$) will be signed only up to the budget.

Finalizing and Executing Transactions. Upon receiving a valid certificate Cert, the validator broadcasts the certificate to other validators, executes the transaction to update the counter's state locally, and sends an *effects signature* to the user. Unlike standard owned objects where the effects signature contains the new state of the object, for a bounded counter object, the effect signature simply marks the transaction as executed because the validator does not know the counter's correct state yet due to its partial view of other certified transactions. Once the user receives $2f+1$ effects certificates, it considers the transaction finalized.

Version Update Requests. When the validator receives a version update request, it processes the request only if all the transactions to which the request points (v.PrevTxs) are certified (line 28). Moreover, all the transactions in v.PrevTxs must have the same version as the validator's current version (line 30). If so, the validator updates its budget. For each certified transaction in v.PrevTxs, the validator increases for additions or decreases for subtractions its budget by η times the transaction's value (line 33). The validator also reclaims its spent budget for transactions it signed and are now included in v.PrevTxs (line 35), ensuring that budget is not deducted twice. After the update, the validator's budget is η times the remaining balance

Algorithm 1 Bounded counter for single owner (validator logic)

```
1:  ▷ Initialize a bounded counter object
2:  ▷ Initial version is v_0 and budget is computed from initial balance
3:  procedure InitBC(Bal_0)
4:      Version ← v_0                          ▷ current version for which the validator signs txs
5:      Bud ← η ∗ Bal_0                        ▷ remaining budget for signing txs for Version
6:      SignedTxs ← ∅                          ▷ set of txs signed so far

7:  ▷ Executed upon receiving a transaction
8:  procedure ProcessTx(Tx)
9:      ▷ Ensure same transaction is not proceesed twice
10:     if Tx ∈ SignedTxs then return sign(Tx)
11:     require valid(Tx)                      ▷ check Tx has valid signatures
12:     require Tx.Version = Version
13:     require Bud + Tx.δ ≥ 0
14:     ▷ Decrease the budget if decrement transaction
15:     if Tx.δ < 0 then Bud ← Bud + Tx.δ
16:     SignedTxs ← SignedTxs ∪ {Tx}
17:     return sign(Tx)

18: ▷ Upon receiving valid certificate signed by 2f + 1 validators; ensure same certificate
    not processed twice
19: procedure ProcessCert(Cert)
20:     broadcast(Cert)
21:     wait until areExecuted(Cert.Tx.Version.PrevTxs)
22:     if valid(Cert) then exec(Cert.Tx)      ▷ execute tx, persist object's state
23:     return sign(Cert)                      ▷ send signature to finalize Cert.Tx to user

24: ▷ Upon receiving version update request signed by the user
25: ▷ Ensure same request is not processed twice
26: procedure ProcessVersionUpdateReq(v)
27:     broadcast(v)
28:     require v.PrevVersion = Version and areCertified(v.PrevTxs)
29:     ▷ Check all txs in v.PrevTxs have version PrevVersion
30:     require allVersionsMatch(v.PrevTxs, v.PrevVersion)
31:     ▷ Update budget based on txs in v.PrevTxs
32:     for Tx in v.PrevTxs do
33:         Bud ← Bud + η ∗ Tx.δ
34:         ▷ Regain spent budget for txs included in v.PrevTxs
35:         if Tx ∈ SignedTxs and Tx.δ < 0 then Bud ← Bud − Tx.δ
36:     ▷ Start signing txs for the updated version
37:     Version ← v
```

of the bounded counter after executing all certified transactions minus the values of the transactions that the validator signed but were not included in v.PrevTxs. Finally, the validator updates its current version to v (line 37).[1]

Honest Users. The bounded counter protocol allows a user to get transactions certified concurrently under low contention, i.e., when the number of concurrent transactions is low. In Alg. 2, we specify how an honest user interacts with the bounded counter. In summary, the honest user ensures the following:

(1) The user sends transactions with the same Version until it sends a version update request (Alg. 2 line 11)
(2) For any version, the user does not send transactions exceeding the budget for that version (Alg. 2 line 10).
(3) Upon exhausting the budget for a version (Alg. 2 line 8), the user sends a version update request containing pointers to all transactions it sent for that version (Alg. 2 line 16).
(4) When the budget falls to 0 for a certain version (perhaps due to rounding), the user sends a transaction requesting to convert the bounded counter object to a standard owned object so that the small amount of remaining balance may be spent through a classic owned object transaction.

[1] In reality, the version identifier stored by the validator and included in each transaction can be a hash of the version update. Collision resistance will bind each version identifier to a unique version update request.

Algorithm 2 Bounded counter for single owner (user logic)

```
1:  procedure Init(Bal_0)
2:      Version ← v_0
3:      Bud ← η ∗ Bal_0
4:      sentTxs ← ∅

5:  ▷ Invoked when application requires updating the bounded counter
6:  procedure Update(δ)
7:      if Bud + δ < 0 then                    ▷ If update exceeds the budget
8:          VersionUpdate()
9:      ▷ Abort if update exceeds the budget even after version update
10:     if Bud + δ < 0 then return Error
11:     Tx ← {Version: Version, δ: δ}
12:     if δ < 0 then Bud ← Bud + Tx.δ
13:     sendToValidators(Tx)
14:     sentTxs ← sentTxs ∪ {Tx}

15: procedure VersionUpdate()
16:     v ← {PrevVersion : Version, PrevTxs : sentTxs}
17:     for Tx in sentTxs do
18:         Bud ← Bud + η ∗ Tx.δ
19:         if Tx.δ < 0 then Bud ← Bud − Tx.δ
20:     ▷ Send a version update request if there is enough remaining budget, otherwise
    request to convert the bounded counter to an owned object
21:     if Bud ≥ minBudget then sendToValidators(v)
22:     else sendToValidators(convertToOwnedObject(sentTxs))
23:     Version ← v
```

An honest user can satisfy these requirements by keeping track of the current version and the validator's budget, all of which can be computed based on the transactions it has sent (see Alg. 2), without any communication. Since an honest user's concurrent transactions do not exceed the initial budget for that version (Item 2), no honest validator exhausts its budget, and the honest user eventually receives signatures from $2f + 1$ validators on all its transactions. After spending this budget, the honest user issues a version update request (Item 3) so that all validators eventually update their budget. This ensures liveness (Def. 4) for honest owners in an asynchronous network (formal proof in Sec. B).

4 Concurrency with Multiple Owners

So far, we have seen designs to improve concurrency for transactions with owned objects. However, these techniques and prior consensus-less systems require that all owned objects accessed in a transaction have the same owner. Stingray enhances object programmability on the fast path, allowing for multi-owner transactions. This, however, opens up the risk of deadlock. We mitigate this risk with our novel FastUnlock protocol.

4.1 Multi-Owner Transactions

All prior fast-path systems require owned objects in a transaction to be owned by the same address. Stingray lifts this restriction: a transaction can reference owned objects from multiple owners. Validators must still ensure that each owned object referenced by a transaction is correctly authorized before signing a transaction. For example, consider an atomic swap transaction that takes object A owned by Alice and object B owned by Bob and exchanges their ownership. For the transaction to be authorized, we need two signatures over the full transaction (or a hash), one from an authorized signer of A (i.e., Alice) and one from an authorized signer of B (i.e.,

Bob). Following the model from [24], Alice and Bob coordinate out-of-band to collect authorizations for a multi-owner transaction. This is akin to how multi-sig blockchain wallets work in practice.

However, enabling multi-owner transactions makes Stingray more susceptible to owned objects being locked through error or malicious behavior. For example, consider the previous scenario. If Alice signs Tx first, Bob may refuse to sign, denying Alice access to her object. If Alice loses patience and tries to use A in another transaction Tx′, then Bob may sign Tx and race Alice's attempt to build a certificate. Now both Tx and Tx′ contain A and conflict which can lead to A (and B) being locked. Sui unlocks such objects after one day, at epoch change. But multi-owner transactions in Stingray make such conflicts more likely, so the latency of one day is unacceptable. Thus, we develop the FastUnlock protocol described in Sec. 4.3 that provides a resolution in seconds.

4.2 Fast Unlock Protocol

As discussed in Sec. 4, concurrency for multi-owner transactions does not come free as it increases the chance that transactions diverge the view of the validators, leading to a loss of liveness. This is already a problem in Sui [30] for clients that run buggy software and may issue conflicting transactions on an owned object, i.e., transactions operating on the same version of an owned object (cf. Sec. 2.1). Their current solution is to periodically run an atomic snapshot sub-protocol as part of an epoch change protocol and then cancel all such conflicting transactions. As a result, at the start of the new epoch, all validators have exactly the same state for all owned objects, and the locked objects can be safely accessed again. In Sui, epoch changes occur once per day. Since multi-owner transactions and collective objects make locks more likely, the latency of one day is unacceptable. To remedy this issue, Stingray introduces a FastUnlock functionality. On a high level, FastUnlock is a merge protocol for conflicting forks in the transaction history of an owned object. Its basic functionality is to identify if there is a possible finalized transaction that has not yet been universally accepted and to force every single validator to accept it as true, dropping any locks on conflicting transactions. The baseline protocol is more conservative and could result in never committing anything. For this reason, in Sec. 4.4 we show how a resolution transaction can be provided along the unlock request, resulting in guaranteed progress.

4.3 Baseline FastUnlock Protocol

For simplicity, we show how the user can unlock a single object by executing a no-op or adopting one of the conflicting transactions. Appendix 4.4 extends the basic protocol to execute a new transaction instead of a no-op. Additionally, we describe at the end of this section how validators can detect two conflicting transactions in the system and *automatically* trigger an unlock.

New Persistent Data Structures. Stingray validators maintain a set of persistent tables abstracted as key-value maps, with contains, get, and set operations. The table

$$\textsc{LockDb}[\mathsf{ObjectKey}] \rightarrow \mathsf{Cert} \text{ or } \textbf{None}$$

maps $\mathsf{ObjectKey} = (\mathsf{ObjectId}, \mathsf{Version})$, an object's identifier and version, to a certificate Cert, or **None** if the object's version exists

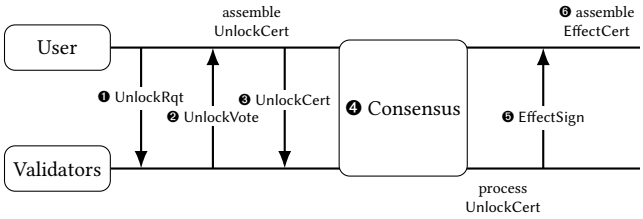

Figure 4: FastUnlock interactions between a user and validators to unlock an object.

Algorithm 3 Process unlock requests

```
▷ Handle UnlockRqt messages from users.
procedure ProcessUnlockTx(UnlockRqt)
    ▷ Check (3.1): Check Auth. (Sec. 4.3)
    if !valid(UnlockRqt) then return error
    ▷ Step (3.2): Check for certificates.
    ObjectKey ← UnlockRqt.ObjectKey
    Cert ← LockDb[ObjectKey]                    ▷ can be None
    ▷ Step (3.3): Record the decision to unlock.
    UnlockVote ← sign(UnlockRqt, Cert)
    UnlockDb[ObjectKey] ← Unlocked
    return UnlockVote
```

but the validator does not hold a certificate for it. The map

$$\textsc{UnlockDb}[\mathsf{ObjectKey}] \rightarrow \textbf{Unlocked}, \textbf{Confirmed}, \text{ or } \textbf{None}$$

records whether a transaction over the specified object version is involved in a current FastUnlock instance (**Unlocked**), has been sequenced by consensus (**Confirmed**), or none of the above (**None**).

All new owned object entries start with UnlockDb[ObjectKey] set to **None**. Once a transaction certificate is sequenced through consensus, it is always executed (whether it is for a shared object transaction or an owned-object-only transaction) and all owned object entries have UnlockDb[ObjectKey] set to **Confirmed**.

FastUnlock Protocol Description. To safely unlock an object, the user interactively constructs a proof, called a *no-commit certificate*, that no transaction modifying that object has been committed or will be committed on the fast path. This proof consists of a message signed by a quorum of validators attesting that they have not already executed a transaction on ObjectKey, and promising that they will not execute any transaction on ObjectKey in the fast path. After that, only certificates sequenced over consensus may affect such an ObjectKey.

Fig. 4 shows the FastUnlock protocol allowing a user to instruct validators to unlock a specific object. A user creates an *unlock request* specifying the object they wish to unlock:

$$\mathsf{UnlockRqt}(\mathsf{ObjectKey}, \mathsf{Auth})$$

This message contains the object's key ObjectKey to unlock (accessible as UnlockRqt.ObjectKey) and an authenticator Auth ensuring the user is authorized to unlock ObjectKey. The authenticator is composed of two parts: (i) a transaction that mutates the object in question (and potentially additional objects) which is signed by the object owner, and (ii) a proof that the party requesting the unlock can modify the object in question. The authenticator prevents rogue unlock requests for objects that are either not under contention

(the transaction shows there exists a transaction that uses the object) or from parties not authorized to act on the objects. The user broadcasts this UnlockRqt message to all validators (Fig. 4 ❶).

Each validator handles the UnlockRqt as follows (Alg. 3). A validator first checks (**Check (3.1)**) the validity of UnlockRqt by verifying the authenticator Auth. Specifically, Auth must contain a valid transaction including ObjectKey, and a signature on the transaction by the owner of ObjectKey. Otherwise, the validator stops processing. The validator attempts to retrieve a certificate Cert for a transaction on ObjectKey if it exists (**Step (3.2)**), or sets Cert to **None**. Then, the validator records that the object in UnlockRqt can only be included in transactions in the consensus path (Alg. 3) by setting its entry in UnlockDb[ObjectKey] to **Unlocked** (**Step (3.3)**). It finally returns a signed *unlock vote* UnlockVote to the user:

$$\mathsf{UnlockVote}(\mathsf{UnlockRqt}, \textbf{Option}(\mathsf{Cert}))$$

This message contains the authorized UnlockRqt and the certificate Cert for some transaction consuming ObjectKey that the validator executed (Fig. 4 ❷). If the validator has not executed any transaction on ObjectKey, then Cert = **None**.

$$\mathsf{UnlockCert}(\mathsf{UnlockRqt}, \textbf{Option}(\mathsf{Cert})).$$

There are two cases in the creation of UnlockCert:

(1) At least one UnlockVote carries a certificate. This scenario indicates that a correct validator has already executed a transaction, which implies that the object is not locked. However, this is not a proof of finality and subsequent steps may invalidate it.
(2) No UnlockVote carries a certificate. This scenario is a 'no-commit' proof as there are $f + 1$ honest validators that will not process certificates (UnlockDb holds **Unlocked**), thus no certificate will be executed in the fast path.

The user submits this UnlockCert for sequencing by the consensus engine (❸).

Algorithm 4 Process unlock certificates

```
▷ Handle UnlockCert message from consensus.
procedure ProcessUnlockCert(UnlockCert)
    ▷ Check (4.1): Check no transaction already processed (Sec. 4.3).
    if UnlockDb[ObjectKey] = Confirmed then return
    ▷ Check (4.2): Check cert validity (Sec. 4.3).
    if !valid(UnlockCert) then return error
    ▷ Execute Cert or None (4.3).
    Cert ← UnlockCert.Cert
    if Cert ≠ None then Tx ← Cert.Tx
    else Tx ← No-Op
    EffectSign ← exec(Tx, UnlockCert)
    ▷ Prevent execution overwrite.
    UnlockDb[ObjectKey] ← Confirmed
    return EffectSign
```

All correct validators observe a consistent sequence of UnlockCert messages output by consensus (❹) and process them in order as follows (Alg. 4). A validator performs the following checks and if any fail, they ignore the certificate:

- **Check (4.1)** They ensure they did not already process another transaction to completion (i.e. UnlockDb is not **Confirmed**) or a different UnlockCert for the same ObjectKey.
- **Check (4.2)** They check UnlockCert is valid, that is, (i) it is correctly signed by a quorum of authorities, and (ii) the certificate Cert it contains is valid or **None**.

The validator then executes the transaction referenced by Cert (Step 4.3) if one exists. Otherwise, if Cert is **None**, the validator undoes any transaction locally executed on the object[2], then executes a no-op, that is, the object contents remain unchanged but its version number increases by one. The validator finally marks every object key as **Confirmed** to prevent future unlock certificates or checkpoint certificates from overwriting execution (Alg. 4) and returns an EffectSign to the user (❺). The user assembles a quorum of $2f + 1$ EffectSign messages into an *effect certificate* EffectCert that determines finality (❻).

Sec. 4.5 details the use of gas objects in the context of FastUnlock and Appendix Sec. B.3 proves the safety and liveness of the Stingray system using FastUnlock. The key insight is that an UnlockCert forces transactions on the owned object to go through consensus. There, either a transaction certificate or an unlock certificate will be sequenced first and executed. If a transaction is finalized, an unlock certificate will always cause the execution of that transaction.

Auto-Unlock. The basic FastUnlock scheme presumes that the request to unlock an object is authenticated by the owner(s) of the object. This ensures that only authorized parties can interfere with the completion of a transaction, but it also restricts who can initiate unlocking in case of loss of liveness. Alternatively, an 'AutoUnlock' can be issued by validators if the fast-path protocol is embedded in the consensus protocol, as proposed by Mysticeti [4]. In such a protocol, the presence of conflicting transactions in the causal history of a consensus block is evidence of loss of liveness. Upon seeing such evidence, validators can start locally processing a virtual unlock request posting the signed unlock requests as transactions in the consensus protocol and forming unlock certificates.

4.4 Contention Mitigation

The basic FastUnlock protocol speeds up recovery from loss of liveness due to mistakes. However, Stingray aims to support workloads on the fast path that are truly under contention. In this case, the basic protocol in Sec. 4.2 is insufficient, since it can result in multiple rounds of locking and no-op unlocking without any user transaction being committed. We present a protocol that proposes a new transaction during the unlock phase that is executed once the unlock is sequenced, ensuring liveness.

In the following protocol, we additionally allow users to unlock multiple objects at once. The multi-object unlock protocol follows the same general flow as the single-object unlock protocol described in Sec. 4.2. We now describe steps ❶-❻ depicted in Fig. 4 for the multi-unlock protocol.

Protocol description. The user first creates an *unlock request* specifying a set of objects to unlock:

$$\mathsf{UnlockRqt}([\mathsf{ObjectKey}], \mathsf{Tx}, \mathsf{Auth})$$

This message contains a list of the object's keys [ObjectKey] to unlock (accessible as UnlockRqt.ObjectKeys), a new transaction

[2]The UnlockCert with Cert being **None** ensures such execution could not have been finalized; only a single layer of execution can ever be undone, and no cascading aborts can happen.

Algorithm 5 Process unlock requests (multi-object)

```
▷ Handle UnlockRqt messages from clients.
procedure ProcessUnlockTx(UnlockRqt)
    ▷ Check (5.1): Check authenticator.
    if !valid(UnlockRqt) then return error
    ▷ Collect certificates.
    c ← None
    for ObjectKey ∈ UnlockRqt.ObjectKeys do
        c ← c ∪ LockDb[ObjectKey]
    UnlockVote ← sign(UnlockRqt, c)
    ▷ Record the decision to unlock.
    if c == None then
        for ObjectKey ∈ UnlockRqt.ObjectKeys do
            UnlockDb[ObjectKey] ← Unlocked
    return UnlockVote
```

Tx to execute if the unlock process succeeds, and an authenticator Auth ensuring the sender is authorized to access all objects in [ObjectKey]. The user broadcasts this message to all validators (❶).

Alg. 5 describes how each validator handles this unlock request UnlockRqt. They first perform Check (5.1) Alg. 5 to check the authenticator Auth is valid with respect to all objects. This check ensures that the user is authorized to mutate all the objects referenced by UnlockRqt and to lock all owned object referenced by Tx. The validator then collects any certificates for the objects referenced by UnlockRqt (Alg. 5) and adds them to the response as Cert. The validator then marks the object in UnlockRqt as reserved for transaction execution through consensus only (Alg. 5).

The validator finally returns an *unlock vote* UnlockVote to the user:

$$\mathsf{UnlockVote}(\mathsf{UnlockRqt}, [\textbf{Option}(\mathsf{Cert})])$$

This message contains the unlock message UnlockRqt itself and possibly a set of certificates [Cert] on transactions including the object keys referenced by UnlockRqt (possible empty) (❷). If Cert is not empty the certified transactions may have been finalized, and should be executed instead of the new transaction.

The user collects a quorum of $2f + 1$ UnlockVote over the same UnlockRqt message and assembles them into an *unlock certificate* UnlockCert:

$$\mathsf{UnlockCert}(\mathsf{UnlockRqt}, \mathsf{Cert})$$

where UnlockRqt is the user-created certified unlock message and UCert is the unions of all set of certificates received in UnlockRqt responses. The user submits this message to the consensus engine (❸) The consensus engine sequences all UnlockCert messages; all correct validators observe the same output sequence (❹).

Alg. 6 describes how validators process these UnlockCert messages after they are sequenced by the consensus engine. The validator first ensures they did not already process another UnlockCert or Cert through checkpoint for the same objects keys (Alg. 6). They then check UnlockCert is valid, that is, the validator ensures (i) it is correctly signed by a quorum of authorities, and (ii) that all certificates [Cert] it contains are valid (Alg. 6). The validator can only execute the transaction Tx specified by the user if UnlockCert.Cert is empty (Alg. 6). The validator then marks every object key of [ObjectKey] as **Confirmed** to prevent any future unlock requests on the ObjectKey from overwriting execution with a different transaction (Alg. 6) and returns a set of EffectSign to the user (❺).

The user assembles an EffectSign from a quorum of $2f + 1$ validators into an *effect certificate* EffectCert that determines finality (❻).

Algorithm 6 Process unlock certificates (multi-object)

```
▷ Handle UnlockCert messages from consensus.
procedure ProcessUnlockCert(UnlockCert)
    ▷ Check (6.1): Check no transaction already processed.
    for ObjectKey ∈ UnlockCert.ObjectKeys do
        if UnlockDb[ObjectKey] = Confirmed then
            return
    ▷ Check (6.2): Check message validity.
    if !valid(UnlockCert) then return error
    ▷ Check (6.3): Can we execute the tx?
    v ← [ ]
    if UnlockCert.Cert = [ ] then
        Tx ← UnlockCert.UnlockRqt.Tx
        EffectSign ← exec(Tx, UnlockCert)
        v ← EffectSign
        for ObjectKey ∈ UnlockCert.ObjectKeys do
            UnlockDb[ObjectKey] = Confirmed
    else
        for Cert ∈ UnlockCert.Cert do
            EffectSign ← exec(Cert)
            v ← v ∪ EffectSign
            for ObjectKey ∈ Cert.ObjectKeys do
                UnlockDb[ObjectKey] = Confirmed
    return v
```

4.5 Handling Gas Objects

Typical transactions not only mutate objects but also consume a gas object to pay for the computation. If, however, the transaction is equivocated then this gas is locked as well. For this reason Stingray requires a fresh gas-object in order for consensus to process the unlock request. Specifically together with Alg. 3, the parties should provide a fresh gas object for their request. This gas object is checked for validity along with the check in Alg. 3 and locked for the unlock transaction in Alg. 3. When the user collects the no-commit proof in the second step of the protocol, the $2f + 1$ collected signatures also serve as a certificate for the gas object. The consensus then checks the validity of the certificate and spends it locally before entering Alg. 4. Then when consensus executes the transaction, one of three scenarios may happen:

- The unlock request is valid and includes a certificate. Then the execution happens as usual and both the gas object for the unlock and the gas object for the execution are consumed.
- The unlock request is valid and comes with a no-op. Then the gas object for unlock is consumed. If there was some locked transaction racing the FastUnlock then the accompanying gas object is potentially blocked. The user can then explicitly unlock that gas object by running FastUnlock.
- The unlock request is not processed because a checkpoint certificate already executed a transaction. Then the gas object is still consumed without altering the state of the ObjectKey.

Note that if gas objects are implemented using bounded counters, the same gas object can be spent concurrently for the transaction and the unlock, thus the above problem wouldn't exist.

5 Implementation

We base our implementation of Stingray on Sui [30] as it is, to our knowledge, the only blockchain currently supporting consensusless transactions. Specifically, we fork the research codebase of Mysticeti [31], which is a fork of the production codebase of Sui, but without irrelevant features such as Admission control, RPC endpoints, support for light clients, enforcement of correct genesis,

etc. Our implementation only modifies the block and transaction processing logic by adding the bounded counter, keeping the networking, storage, and cryptography layers untouched. We open-source our implementation of Stingray and our orchestration tools to ensure reproducibility of our results[3].

6 Evaluation

We evaluate the throughput and latency of Stingray through experiments conducted on Amazon Web Services (AWS), demonstrating its performance improvements over the state-of-the-art.

We compare Stingray with the consensus-less fast path of Sui [30], called Mysticeti-FPC [4], as to our knowledge, Sui is the only blockchain supporting consensus-less transactions. We did not compare with other consensus-less systems, including FastPay [9], Astro [19], or Zef [10] because they only support payments and are thus not adapted to showcase loads under high concurrency[4]. Furthermore, these systems lack a mechanism to unlock transactions and thus cannot optimistically handle contention.

Our evaluation demonstrates the following claims:

- **C1**: Clients of Stingray submitting a commutative load experience a lower latency and higher throughput than those of the Sui baseline.
- **C2**: There is no noticeable performance difference between Sui using owned objects and Stingray using bounded counters for loads with no contention (i.e., parallel). In other words, there is no performance trade-off in adopting Stingray.
- **C3**: Operations under (crash) faults do not overly penalize Stingray in comparison to Sui. That is, both systems observe similar performance degradation when validators are faulty.

Note that evaluating the performance of BFT protocols in the presence of Byzantine faults is an open research question [7], and relies on formal proofs.

6.1 Experimental Setup

We deploy all systems on a geo-distributed network of validators, each running on a dedicated machine. Sec. C details the precise machine specs, validator configuration, and the network setup.

In the following graphs, each data point is the p50 latency and the error bars represent the p90 latency (error bars are sometimes too small to be visible on the graph). We instantiate several geo-distributed benchmark clients within each validator, submitting transactions at a fixed rate for 5 minutes. We increase the load of transactions sent to the systems and record the throughput and latency. As a result, each plot illustrates the 'steady state' latency of the system under low load and the maximum throughput it can serve after which latency grows steeply. Transactions in the benchmarks contain 512 bytes. The ping latency between the validators varies from 50ms to 250ms.

By *latency*, we mean the time between when the client submits the transaction and when the transaction is finalized by the validators. By *throughput*, we mean the number of transactions finalized per second during the run.

[3] https://github.com/asonnino/mysticeti/tree/stingray (commit `0ae4bb5`)

[4] Parallelizing payments issued from an account can be achieved by splitting the available balance into multiple accounts before initiating the payments.

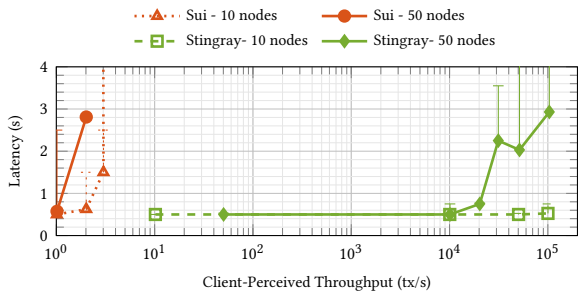

Figure 5: Comparing throughput and latency of Sui and Stingray with a *commutative* load. WAN measurements with 10 and 50 validators.

Protocols	1 tx	10 tx	100 tx
Sui	400-500ms	4-5s	≈ 50s
Stingray	< 500ms	< 500ms	< 500ms

Table 1: Average total time required to submit a load of 1, 10, and 100 commutative transactions to Sui and Stingray.

6.2 Benchmark under Commutative Load

Fig. 5 compares the throughput and latency experienced by clients of Sui and Stingray submitting a load of *commutative* transactions. In Sui, these transactions are implemented using operations on the same owned object [30], whereas, in Stingray, they rely on bounded counter withdrawals with values much lower than the available balance (Sec. 3). Specifically, we measure the maximum rate at which a client can submit these transactions and the corresponding end-to-end latency. Both systems operate in a failure-free wide-area network (WAN) environment, configured with committees of 10 and 50 validators to reflect small and large committees.

As expected, Sui clients can submit only about two commutative transactions per second. This is because Sui fails to exploit the commutativity of these transactions and processes them sequentially because it detects false dependencies based on memory access patterns. Consequently, a client must wait approximately 500ms for one transaction to complete before submitting the next. Despite Sui's low baseline latency [4, 30], this commutative load results in a latency proportional to the number of transactions submitted. Consequently, clients perceive significantly higher overall latency as the transaction count increases. For example (see Tab. 1), a single transaction incurs the state-of-the-art latency of 500 ms, but submitting 100 of these transactions causes the total latency to grow linearly to 50 seconds.

In contrast, Fig. 5 demonstrates that Stingray enables parallel submission of commutative transactions, maintaining latencies under 500 ms. This performance holds even for workloads of 10-20k commutative transactions with a large committee of 50 validators (note the log scale on the x-axis) or 100k transactions with a smaller committee of 10 validators. This improvement stems from Stingray's use of the bounded counter, which processes transactions concurrently. As a result, transaction latency remains unaffected by the submission rate until the system reaches saturation. Throughout

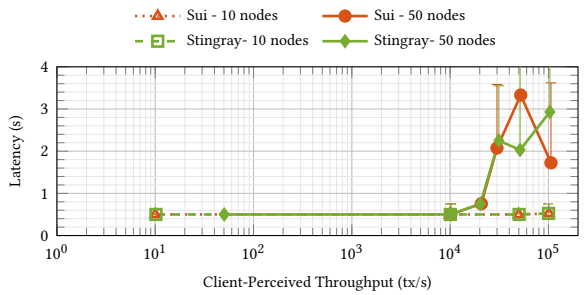

Figure 6: Comparing throughput and latency of Sui and Stingray with a *parallel* load. WAN measurements with 10 and 50 validators.

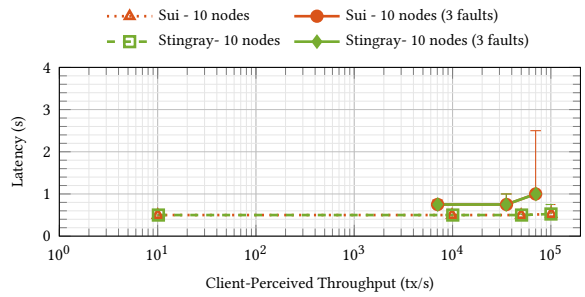

Figure 7: Comparing throughput and latency of Sui and Stingray with a *parallel* load. WAN measurements with 10 validators, 3 faults.

these benchmarks, the CPU utilization of the validators of both systems remains roughly below 20% and the validators consume less than 10GB of memory (when experiencing the highest loads).

These results validate our claim **C1**: clients of Stingray experience lower latency and higher throughput than those of Sui when handling commutative transaction loads.

6.3 Benchmark under Parallel Load

Fig. 6 compares the throughput and latency experienced by clients of Sui and Stingray submitting a load of *independent* transactions. Both systems operate in a failure-free wide-area network (WAN) environment, configured with committees of 10 and 50 validators.

In contrast to the previous benchmark, the transactions in this benchmark are implemented using operations on different owned objects. As a result, both systems process these transactions concurrently, and the throughput and latency experienced by the clients of both systems are similar. In both cases, clients experience a latency of less than 500 ms and a throughput of 10-20k transactions per second with a large committee of 50 validators and 100k transactions per second with a small committee of 10 validators.

This result validates our claim **C2**: there is no noticeable performance difference between Sui and Stingray for loads with no contention (in this case, both systems use owned objects), i.e., there is no performance trade-off in Stingray.

6.4 Benchmark under Faults

Fig. 7 compares the throughput and latency experienced by clients of Sui and Stingray submitting a load of independent transactions when a committee of 10 validators experiences 3 (crash) faults, which is the maximum number of faults that can be tolerated in this system's configuration. The results show that both systems observe similar performance degradation when validators are faulty. In both cases, the throughput drops to about 70k transactions per second, and the latency increases to about 1 second. This result validates our claim **C3**: operations under (crash) faults do not overly penalize Stingray in comparison to Sui.

7 Related Work

Stingray is closely related to three research directions: consensus-less (fast path) blockchains, parallel execution engines, and replicated data types. Consensus-less blockchains were originally developed for payments (both in theory [3, 19, 24, 29] and practice [9, 37]) achieving minimal latency. These systems, however, support only payments without programmability and lack validator reconfiguration capabilities. Groundhog [40] similarly avoids consensus for commutative transactions but cannot process non-commutative ones. The Sui Lutris system [12], implemented in the Sui blockchain [30], integrates FastPay [9] with the Bullshark [43] consensus protocol to offer both low-latency payments and full programmability. Mysticeti [4] further enhances this approach by eliminating Sui's redundant broadcasting and reducing signature verification costs.

Parallel execution in blockchains is a relatively new research area led by Solana [45] and FuelVM [22]. On the research side, Block-STM [23] focuses on shared memory and executes blocks of transactions instead of streaming. This creates a tension between high throughput and low latency as high throughput needs a high batch size, but collecting this batch increases the latency. On the other hand, Pilotfish [28] focuses on multi-machine execution and follows Sui's streaming architecture. Sui also supports parallel execution in a single machine but only for transactions accessing different memory locations. Stingray improves upon the state of the art by enabling the parallel execution of transactions that access the same memory location, as long as they do not conflict. While this paper emphasizes parallelization on the consensus-less path, the same principles apply even more easily post-consensus as there is no risk of losing liveness due to equivocation.

Another closely related research area is replicated data types. Unlike conflict-free data types (CRDTs) [32, 41], our bounded counter supports non-commutative and non-inflationary state transitions. While some previous work [2, 13–15, 17, 21, 26, 36, 46] developed Byzantine fault-tolerant CRDTs and others developed bounded counters that are safe under no faults [8, 38] and crash faults [6], our work achieves the best of both through the first Byzantine fault-tolerant bounded counter. CRDTs with non-commutative operations inherently require coordination among the replicas [5, 11, 27], even without Byzantine faults. Accordingly, our bounded counter

also requires coordination but the number of rounds of coordination is at most logarithmic in the counter's initial value.

Finally, Stingray requires smart contracts to manually specify each object type. However, it is compatible with static analysis tools such as CoSplit [39], which automatically infer ownership and commutativity summaries from smart contracts.

Acknowledgments

This work is sponsored by Mysten Labs. This work was done partly when Srivatsan Sridhar was interning with Mysten Labs. We thank George Danezis for initial discussions and feedback and Igor Zablotchi, David Tse, and Roger Wattenhofer for feedback.

References

[1] Paulo Sérgio Almeida and Carlos Baquero. 2019. Scalable eventually consistent counters over unreliable networks. *Distributed Comput.* 32, 1 (2019), 69–89.
[2] Paulo Sérgio Almeida and Ehud Shapiro. 2024. The Blocklace: A Byzantine-repelling and Universal Conflict-free Replicated Data Type. arXiv:2402.08068v4 [cs.DC]
[3] Alex Auvolat, Davide Frey, Michel Raynal, and François Taïani. 2020. Money Transfer Made Simple: a Specification, a Generic Algorithm, and its Proof. *Bull. EATCS* 132 (2020).
[4] Kushal Babel, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, Alberto Sonnino, and Mingwei Tian. 2025. Mysticeti: Reaching the Latency Limits with Uncertified DAGs. In *NDSS*. The Internet Society.
[5] Peter Bailis, Alan D. Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2014. Coordination Avoidance in Database Systems. *Proc. VLDB Endow.* 8, 3 (2014), 185–196.
[6] Valter Balegas, Diogo Serra, Sérgio Duarte, Carla Ferreira, Marc Shapiro, Rodrigo Rodrigues, and Nuno M. Preguiça. 2015. Extending Eventually Consistent Cloud Databases for Enforcing Numeric Invariants. In *SRDS*. IEEE Computer Society, 31–36.
[7] Shehar Bano, Alberto Sonnino, Andrey Chursin, Dmitri Perelman, Zekun Li, Avery Ching, and Dahlia Malkhi. 2021. Twins: BFT Systems Made Robust. In *OPODIS (LIPIcs, Vol. 217)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 7:1–7:29.
[8] Daniel Barbará and Hector Garcia-Molina. 1994. The Demarcation Protocol: A Technique for Maintaining Constraints in Distributed Database Systems. *VLDB J.* 3, 3 (1994), 325–353.
[9] Mathieu Baudet, George Danezis, and Alberto Sonnino. 2020. FastPay: High-Performance Byzantine Fault Tolerant Settlement. In *AFT*. ACM, 163–177.
[10] Mathieu Baudet, Alberto Sonnino, Mahimna Kelkar, and George Danezis. 2023. Zef: Low-latency, Scalable, Private Payments. In *WPES@CCS*. ACM, 1–16.
[11] Rida A. Bazzi and Sara Tucci Piergiovanni. 2024. The Fractional Spending Problem: Executing Payment transactions in parallel with less than $f+1$ validations. In *PODC*. ACM, 295–305.
[12] Sam Blackshear, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Xun Li, Mark Logan, Ashok Menon, Todd Nowacki, Alberto Sonnino, Brandon Williams, and Lu Zhang. 2024. Sui Lutris: A Blockchain Combining Broadcast and Consensus. In *CCS*. ACM, 2606–2620.
[13] Margarita Capretto, Martín Ceresa, Antonio Fernández Anta, Antonio Russo, and César Sánchez. 2022. Setchain: Improving Blockchain Scalability with Byzantine Distributed Sets and Barriers. In *Blockchain*. IEEE, 87–96.
[14] Margarita Capretto, Martín Ceresa, Antonio Fernández Anta, Antonio Russo, and César Sánchez. 2024. Improving Blockchain Scalability with the Setchain Data-Type. *Distributed Ledger Technol. Res. Pract.* 3, 2 (2024), 12.
[15] Hua Chai and Wenbing Zhao. 2014. Byzantine Fault Tolerance for Services with Commutative Operations. In *IEEE SCC*. IEEE Computer Society, 219–226.
[16] K Mani Chandy and Leslie Lamport. 1985. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems (TOCS)* 3, 1 (1985), 63–75.
[17] Vicent Cholvi, Antonio Fernández Anta, Chryssis Georgiou, Nicolas Nicolaou, Michel Raynal, and Antonio Russo. 2021. Byzantine-Tolerant Distributed Grow-Only Sets: Specification and Applications. In *FAB (OASIcs, Vol. 92)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2:1–2:19.
[18] Circle. 2025. Fully Backed Digital Dollars. https://www.circle.com/usdc.
[19] Daniel Collins, Rachid Guerraoui, Jovan Komatovic, Petr Kuznetsov, Matteo Monti, Matej Pavlovic, Yvonne-Anne Pignolet, Dragos-Adrian Seredinschi, Andrei Tonkikh, and Athanasios Xygkis. 2020. Online Payments by Merely Broadcasting Messages. In *DSN*. IEEE, 26–38.
[20] ethereum.org. 2024. The Complete Guide to Ethereum. https://ethereum.org/en/ Last accessed: Dec 24, 2024.
[21] Davide Frey, Lucie Guillou, Michel Raynal, and François Taïani. 2024. Process-commutative distributed objects: From cryptocurrencies to Byzantine-Fault-Tolerant CRDTs. *Theor. Comput. Sci.* 1017 (2024), 114794.
[22] FuelVM. 2025. Intro | Fuel Docs. https://docs.fuel.network/docs/intro/what-is-fuel/.
[23] Rati Gelashvili, Alexander Spiegelman, Zhuolun Xiang, George Danezis, Zekun Li, Dahlia Malkhi, Yu Xia, and Runtian Zhou. 2023. Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing. In *PPoPP*. ACM, 232–244.
[24] Rachid Guerraoui, Petr Kuznetsov, Matteo Monti, Matej Pavlovic, and Dragos-Adrian Seredinschi. 2019. The Consensus Number of a Cryptocurrency. In *PODC*. ACM, 307–316.
[25] Martin Kleppmann. 2021. Thinking in events: from databases to distributed collaboration software: keynote at the 15th ACM International Conference on Distributed and Event-Based Systems (DEBS). In *DEBS*. ACM, 15–24.
[26] Martin Kleppmann. 2022. Making CRDTs Byzantine fault tolerant. In *PaPoC@EuroSys*. ACM, 8–15.
[27] Martin Kleppmann and Heidi Howard. 2020. Byzantine Eventual Consistency and the Fundamental Limits of Peer-to-Peer Databases. arXiv:2012.00472v1 [cs.DC]
[28] Quentin Kniep, Lefteris Kokoris-Kogias, Alberto Sonnino, Igor Zablotchi, and Nuda Zhang. 2024. Pilotfish: Distributed Execution for Scalable Blockchains. arXiv:2401.16292v3 [cs.DC]
[29] Petr Kuznetsov, Yvonne-Anne Pignolet, Pavel Ponomarev, and Andrei Tonkikh. 2023. Permissionless and asynchronous asset transfer. *Distributed Comput.* 36, 3 (2023), 349–371.
[30] Mysten Labs. 2022. Build Without Boundaries. https://sui.io.
[31] Mysten Labs. 2024. Mysticeti. https://github.com/asonnino/mysticeti.
[32] Shadaj Laddad, Conor Power, Mae Milano, Alvin Cheung, Natacha Crooks, and Joseph M. Hellerstein. 2022. Keep CALM and CRDT On. *Proc. VLDB Endow.* 16, 4 (2022), 856–863.
[33] Linera. 2022. Unlocking the Power of Decentralization. https://linera.io.
[34] Jean-Philippe Martin and Lorenzo Alvisi. 2006. Fast Byzantine Consensus. *IEEE Trans. Dependable Secur. Comput.* 3, 3 (2006), 202–215.
[35] Friedemann Mattern. 1993. Efficient algorithms for distributed snapshots and global virtual time approximation. *Journal of parallel and distributed computing* 18, 4 (1993), 423–434.
[36] Pezhman Nasirifard, Ruben Mayer, and Hans-Arno Jacobsen. 2023. OrderlessChain: A CRDT-based BFT Coordination-free Blockchain Without Global Order of Transactions. In *Middleware*. ACM, 137–150.
[37] Pod Network. 2025. Pod Network. https://pod.network.
[38] Patrick E. O'Neil. 1986. The Escrow Transactional Method. *ACM Trans. Database Syst.* 11, 4 (1986), 405–430.
[39] George Pîrlea, Amrit Kumar, and Ilya Sergey. 2021. Practical smart contract sharding with ownership and commutativity analysis. In *PLDI*. ACM, 1327–1341.
[40] Geoffrey Ramseyer and David Mazières. 2024. Groundhog: Linearly-Scalable Smart Contracting via Commutative Transaction Semantics. arXiv:2404.03201v1 [cs.DC]
[41] Marc Shapiro, Nuno M. Preguiça, Carlos Baquero, and Marek Zawirski. 2011. Conflict-Free Replicated Data Types. In *SSS (LNCS, Vol. 6976)*. Springer, 386–400.
[42] Jakub Sliwinski and Roger Wattenhofer. 2019. ABC: Proof-of-Stake without Consensus. arXiv:1909.10926v3 [cs.CR]
[43] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: DAG BFT Protocols Made Practical. In *CCS*. ACM, 2705–2718.
[44] Matthew Weidner and Paulo Sérgio Almeida. 2022. An oblivious observed-reset embeddable replicated counter. In *PaPoC@EuroSys*. ACM, 47–52.
[45] Anatoly Yakovenko. 2018. Solana: A New Architecture for a High Performance Blockchain v0. 8.13. *Whitepaper* (2018).
[46] Wenbing Zhao. 2016. Optimistic Byzantine fault tolerance. *Int. J. Parallel Emergent Distributed Syst.* 31, 3 (2016), 254–267.

A Open Science

We provide the following artifacts to evaluate our core contributions:

- The implementation of Stingray
- The orchestration scripts to run the benchmarks described in Sec. 6

Both artifacts are available at https://github.com/asonnino/mysticeti/tree/stingray (commit `0ae4bb5`) .

We additionally provide a brief tutorial on how to deploy a testbed and run benchmarks using the provided orchestration scripts.

Deploying a testbed. (Requires having already created AWS credentials.) The file '~/.aws/credentials' should have the following content:

```
[default]
aws_access_key_id = YOUR_ACCESS_KEY_ID
aws_secret_access_key = YOUR_SECRET_ACCESS_KEY
```

configured with account-specific AWS *access key id* and *secret access key*. It is advise to not specify any AWS region as the orchestration scripts need to handle multiple regions programmatically.

A file 'settings.yaml' contains all the configuration parameters for the testbed deployment. We run the experiments of Sec. 6 with the following settings:

```
---
testbed_id: "${USER}-stingray"
cloud_provider: aws
token_file: "/Users/${USER}/.aws/credentials"
ssh_private_key_file: "/Users/${USER}/.ssh/aws"
regions:
  - us-east-1
  - us-west-2
  - ca-central-1
  - eu-central-1
  - eu-west-1
  - eu-west-2
  - eu-west-3
  - eu-north-1
  - ap-south-1
  - ap-southeast-1
  - ap-southeast-2
  - ap-northeast-1
  - ap-northeast-2
specs: m5d.8xlarge
repository:
  url: https://github.com/AUTHOR/REPO.git
  commit: main
node_parameters_path: "crates/orchestrator/assets/node-parameters.yml"
client_parameters_path: "crates/orchestrator/assets/client-parameters.yml"
benchmark_duration: 1000
```

where the file '/Users/$USER/.ssh/aws' holds the ssh private key used to access the AWS instances, and 'AUTHOR' and 'REPO' are respectively the GitHub username and repository name of the codebase to benchmark. Additional configurations can be appended to customize benchmark behavior. To enable fault injection, add the following to 'settings.yaml':

```
faults: !Permanent { faults: 3 }
```

For client-specific settings, append configurations to the file 'client-parameters.yaml'. To enable Bcounters:

```
load_type: !BCounter
  total_budget: 180000000000000
```

To restrict transactions to sequential execution only:

```
load_type: !Sui
  sequential: true
```

The orchestrator binary provides various functionalities for creating, starting, stopping, and destroying instances. For instance, the following command to boots 2 instances per region (if the settings file specifies 13 regions, as shown in the example above, a total of 26 instances will be created):

```
cargo run --bin orchestrator -- testbed deploy --instances 2
```

The following command displays he current status of the testbed instances

```
cargo run --bin orchestrator testbed status
```

Instances listed with a green number are available and ready for use and instances listed with a red number are stopped. It is necessary to boot at least one instance per load generator, one instance per validator, and one additional instance for monitoring purposes (see below). The following commands respectively start and stop instances:

```
cargo run --bin orchestrator -- testbed start
cargo run --bin orchestrator -- testbed stop
```

It is advised to always stop machines when unused to avoid incurring in unnecessary costs.

Running Benchmarks. Running benchmarks involves installing the specified version of the codebase on all remote machines and running one validator and one load generator per instance. For example, the following command benchmarks a committee of 50 validators under a constant load of 1,000 tx/s:

```
cargo run --bin orchestrator -- benchmark \
    --committee 50 fixed-load --loads 1000
```

The nodes and clients configuration files, specified in

`crates/orchestrator/assets/node-parameters.yml`

and

`crates/orchestrator/assets/client-parameters.yml`

are used to respectively parametrize the nodes and clients. We run our benchmarks with the nodes configuration file as follows:

```
leader_timeout:
  secs: 1
  nanos: 0
consensus_only: false
```

and the client configuration file as follows:

```
initial_delay:
  secs: 400
  nanos: 0
```

Monitoring. The orchestrator provides facilities to monitor metrics. It deploys a Prometheus instance and a Grafana instance on a dedicated remote machine. Grafana is then available on the address printed on `stdout` when running benchmarks with the default username and password both set to `admin`. An example Grafana dashboard can be found in the file 'grafana-dashboard.json'[5].

B Proofs

B.1 Security Proof for Bounded counter

Theorem 1. *Alg. 1 satisfies validity.*

Proof. Validators execute only valid transactions with a valid certificate (Alg. 1 line 22). □

Theorem 2. *Alg. 1 satisfies eventual consistency.*

Proof. If one honest validator executes a transaction (Alg. 1 line 22), the validator also broadcasts the certificate for the transaction (line 20). Eventually, all validators also receive the version update request Cert.Tx.Version (along with their certified transactions) corresponding to the certified transaction because at least one honest validator (who signed the certified transaction) received the request and broadcast it (line 27). Thus, eventually, all honest validators will execute the transaction. □

As a warmup for proving global safety, we show how Key Idea 1 ensures that within a single version, any subset of the certified transactions does not spend too much balance.

Definition 6. *The value function is defined on a set of transactions $\mathcal{S}$ as $\Delta(\mathcal{S}) = \sum_{Tx \in \mathcal{S}} Tx.\delta$.*

[5] https://github.com/asonnino/mysticeti/blob/stingray/crates/orchestrator/assets/grafana-dashboard.json (commit `80545ac`)

Lemma 1. *For any version v, let $\overline{Bud}_v$ be the average budget of all honest validators at the time when they set Version to v.*[6] *Let C_v be the set of certified transactions with version v. Then, for all $T \subseteq C_v$: $\Delta(T) \geq -\frac{1}{\eta}\overline{Bud}_v$.*

Proof. Let $C_v^- \subseteq C_v$ be the set of decrement transactions (Tx such that $\mathsf{Tx}.\delta < 0$). It is sufficient to show that $\Delta(C_v^-) \geq -\frac{1}{\eta}\overline{\mathsf{Bud}}_v$.

Suppose that $f_\mathrm{r} \leq f$ validators are adversarial (so, $n - f_\mathrm{r} = 3f + 1 - f_\mathrm{r}$ are honest). Each certified transaction has $2f + 1$ signatures, of which at most f_r are from adversarial validators. All other signatories are honest, thus if $\mathsf{Tx} \in C_v^-$ is certified, at least $(2f + 1 - f_\mathrm{r})|\mathsf{Tx}.\delta|$ is deducted from the total budget of all honest validators (note that $\mathsf{Tx}.\delta < 0$). Suppose, for contradiction, that a set of decrement transactions C_v^- are certified such that $\Delta(C_v^-) < -\frac{1}{\eta}\overline{\mathsf{Bud}}_v$. Then, the average budget of all honest nodes is $\overline{\mathsf{Bud}} \leq \overline{\mathsf{Bud}}_v + \frac{2f+1-f_\mathrm{r}}{3f+1-f_\mathrm{r}}\Delta(C_v^-) \leq \overline{\mathsf{Bud}}_v + \frac{f+1}{2f+1}\Delta(C_v^-) < 0$. This is a contradiction because no honest validator signs a transaction that would cause its budget to fall below 0 (Alg. 1 line 13), so the average budget of honest validators cannot fall below 0. □

Next, we will prove that the version updates introduced as Key Idea 2 preserve global safety. To do this, we first prove that the versions corresponding to certified transactions form a chain in which each version is the PrevVersion of the next (Lem. 2). This follows from a quorum-intersection argument and the fact that the versions of transactions signed by any single honest validator form a chain. This property allows us to define a linearly growing history of the bounded counter containing certified transactions from each version update request's PrevTxs (Def. 8). Using this, we show that the average budget of honest validators at the time they update their local Version to v is η times the balance after executing transactions in the counter's history, minus any budget deducted for certified transactions that were not included in the history (Lem. 3). Combining this with Lem. 1, we prove that the certified transactions across all versions do not spend more than the initial balance (Thm. 3). Since validators only execute certified transactions, this ensures global safety.

Definition 7. *For any version $v \neq v_0$, define the parent version $p(v)$ as $v.PrevVersion$.*

Lemma 2. *Let $\mathcal{V}$ be the set of versions for which there exists at least one certified transaction. If $\mathcal{V} \neq \emptyset$, then $\mathcal{V} = \{v_0, ..., v_{|\mathcal{V}|-1}\}$ such that for all $i = 1, ..., |\mathcal{V}| - 1$, $p(v_i) = v_{i-1}$.*

Proof. If no transactions have been certified, $\mathcal{V} = \emptyset$. If at least one transaction is certified, then $v_0 \in \mathcal{V}$. This is because initially, honest validators sign only transactions with version v_0 (Alg. 1 lines 4 and 12) and will not sign transactions for a different version until they receive a version update request containing certified transactions (line 28). Since a certificate requires at least one honest validator's signature, at least one certified transaction must have version v_0.

For any honest validator j, if it updates Version from v to v', it must be such that $p(v') = v$ (Alg. 1 line 28). Therefore, for all $v \in \mathcal{V}$, there exists a sequence $v_0, ..., v$ in which each version is the parent of the next version. In other words, the versions in $\mathcal{V}$ form a tree rooted at v_0 with parent links as edges.

Now all that remains to show is that this tree is, in fact, a chain. That is, there is no $v, v' \in \mathcal{V}$ such that $v \neq v'$ and $p(v) = p(v')$. This follows from quorum intersection. If there was $v, v' \in \mathcal{V}$ such that $v \neq v'$ and $p(v) = p(v')$, then for both versions v and v', there is a set of $2f + 1$ validators that signed transactions with that version. These two sets of $2f+1$ validators have at least $2(2f+1)-n = f+1$ validators in common (since $n = 3f + 1$). However, since at most f validators are adversarial, at least one honest validator signed both transactions with version v and v'. However, this is a contradiction because once the honest validator signs a transaction for version v, it will never sign a transaction for version v' since there is no path $v, ..., v'$ in which each is a parent of the next one. □

Definition 8. *Define the history H_v of a version as $H_{v_0} = \emptyset$ and $H_{v \neq v_0} = H_{p(v)} \cup v.PrevTxs$.*

Let $C^i = C_{v_1} \cup ... \cup C_{v_i}$ be the set of certified transactions with versions up to v_i. Let $\widetilde{H}_{v_i} = C^i \setminus H_{v_i}$ be the set of certified transactions not included in the history. For any set of transactions $\mathcal{S}$, $\mathcal{S}^- \subseteq \mathcal{S}$ contains transactions Tx such that $\mathsf{Tx}.\delta < 0$, and $\mathcal{S}^+ = \mathcal{S} \setminus \mathcal{S}^-$.

Lemma 3. *For any version $v_i \in \mathcal{V}$, the average budget of all honest validators at the time they upgrade to version v_i satisfies $\overline{Bud}_{v_i} \leq \eta(Bal_0 + \Delta(H_{v_i}) + \Delta(\widetilde{H}_{v_i}^-))$.*

Proof. Throughout the execution, an honest validator i) starts with an initial budget of $\eta\mathsf{Bal}_0$, then ii) decreases its budget for every decrement transaction signed (Alg. 1 line 15), iii) updates its budget for every certified transaction included in a version update request (line 33), and iv) reclaims its budget for every certified decrement transaction it had previously signed that is included in a version update request (line 35). Suppose that $f_\mathrm{r} \leq f$ validators are adversarial (so, $n - f_\mathrm{r} = 3f + 1 - f_\mathrm{r}$ are honest). Combining these four components, the average budget of all honest validators at the time they update to version v_i is

$$\begin{aligned}\overline{\mathsf{Bud}}_{v_i} &\leq \eta\mathsf{Bal}_0 + \frac{2f+1-f_\mathrm{r}}{3f+1-f_\mathrm{r}}\Delta(C^{i^-}) \\ &\quad + \eta\Delta(H_{v_i}) - \frac{2f+1-f_\mathrm{r}}{3f+1-f_\mathrm{r}}\Delta(H_{v_i}^-) \\ &= \eta\big(\mathsf{Bal}_0 + \Delta(H_{v_i})\big) + \frac{2f+1-f_\mathrm{r}}{3f+1-f_\mathrm{r}}\Delta(\widetilde{H}_{v_i}^-) \\ &\leq \eta\big(\mathsf{Bal}_0 + \Delta(H_{v_i}) + \Delta(\widetilde{H}_{v_i}^-)\big).\end{aligned}$$

□

Theorem 3. *The bounded counter protocol (validator code: Alg. 1) satisfies global safety.*

Proof. For any given subset of honest validators, let T be the set of transactions executed by some validator in this subset. Let v_k be the latest version in T. Since validators only execute certified transactions (Alg. 1 line 22), $T \subseteq C^k$. Moreover, validators execute transactions in v_k.PrevTxs before executing transactions with version v_k (Alg. 1 line 21), so $T \supseteq H_{v_k}$.

Recall that we partitioned $C^{k-1} = H_{v_k} \cup \widetilde{H}_{v_k}^- \cup \widetilde{H}_{v_k}^+$, that is, certified transactions with versions up to v_{k-1} may be in the history of version k, and those that are not may be either increments or

[6] Count a validator's budget as 0 if it never sets Version to v.

decrements. Given these constraints, it is sufficient to prove that $\mathsf{Bal}_0 + \Delta(T) \geq 0$ for the worst case $T = H_{v_k} \cup \widetilde{H}^-_{v_k} \cup C^-_{v_k}$ where all decrement transactions and no increment transactions beyond H_{v_k} are executed.

$$\Delta(T) = \Delta(H_{v_k}) + \Delta(\widetilde{H}^-_{v_k}) + \Delta(C^-_{v_k}) \quad (1)$$

$$\geq \Delta(H_{v_k}) + \Delta(\widetilde{H}^-_{v_k}) - \frac{1}{\eta}\overline{\mathsf{Bud}}_{v_k} \quad \text{(Lem. 1)} \quad (2)$$

$$\geq -\mathsf{Bal}_0 \quad \text{(Lem. 3)} \quad (3)$$

□

B.2 Liveness Proof for the Bounded Counter

Theorem 4. *The bounded counter protocol (validator code: Alg. 1, user code: Alg. 2) satisfies liveness. If the user sends only decrement transactions, all transactions sent by an honest user will be executed by honest validators in $O(\log(Bal_0))$ rounds.*

Proof. First, we show that when the user runs Alg. 2, the budget Bud computed by the user matches the budget computed by each honest validator. Let Bud^c_v be the budget computed by the user and let Bud^j_v be the budget computed by a validator j at the time when they each update their local Version to v. We will first show that for all validators j, $\mathsf{Bud}^j_v = \mathsf{Bud}^c_v = \eta(\mathsf{Bal}_0 + H_v)$.

To show this, first note that for every honest validator j, $\mathsf{Bud}^j_v = \eta(\mathsf{Bal}_0 + H_v)$. This can be seen from Lem. 3 along with the observation that $\widetilde{H}^-_v = \emptyset$, i.e., there are no certified transactions not included in the history of the latest version update, because the user includes all transactions it sent in the version update (Alg. 2 line 16). Second, we see that $\mathsf{Bud}^c_v = \eta(\mathsf{Bal}_0 + H_v)$. This is because, as argued above, H_v is exactly the set of transactions ever sent by the user, and for each sent transaction, the user's budget updates in Alg. 2 lines 12, 18 and 19 have the net effect of updating the budget by $\eta * \mathsf{Tx}.\delta$ for each transaction Tx sent by the user.

Finally, for any given version v, the set of decrement transactions $\mathcal{S}$ the user sends satisfies $\mathsf{Bud}^c_v + \Delta(\mathcal{S}) \geq 0$ (Alg. 2 lines 10 and 12). Therefore, every honest validator signs these transactions eventually (once all messages are delivered). If at most f validators are adversarial, every transaction gets certified, and thus eventually executed by honest validators.

Within a given version, all transactions sent by the user get certified concurrently, without any additional rounds of communication. At each version update, validators must wait to execute transactions with the previous version before executing transactions with the new version, thus successive versions are processed sequentially. If the user sends only decrement transactions, at each version update, the budget gets scaled by a factor $\eta < 1$. Thus, after $O(\log(\mathsf{Bal}_0))$ version updates, the budget will fall below a pre-specified bound minBudget, after which the user can convert the bounded counter object to a standard owned object to spend the remaining balance in a single transaction. □

B.3 Security Arguments for FastUnlock

We argue about the safety and liveness of FastUnlock. Intuitively, FastUnlock does not invalidate the finality guarantees of the normal fast path operations. That is, a client holding an effect certificate can be assured that its transaction will never be reverted.

Theorem 5. *If there exists an effect certificate EffectCert over a transaction Tx, the execution of Tx is never reverted.*

Proof. We assume that the execution of Tx is reverted and show a contradiction. The transaction can only be reverted if there exists an UnlockCert carrying an empty certificate over an ObjectKey modified by Tx. From Check (3.2) of Alg. 3 a correct validator only signs an UnlockVote with an empty Cert only if it has not executed anything for ObjectKey. From our assumption that ObjectKey did admit a no-op there should be $f + 1$ honest validators that did not partake in the generation of the EffectCert of Tx and hence passed the check. Additionally, for the EffectCert to exist by definition it has $2f + 1$ signatories over the ObjectKey in question, at least $f + 1$ of them being honest. This implies a total of at least $f+1+f+1+f = 3f + 2 > 3f + 1$ validators, hence a contradiction. □

The converse also applies, that is, if an UnlockCert exists, then no EffectCert over the ObjectKey will be generated in the fast path. The proof works analogously by adding an extra check during EffectCert generation in which correct validators refuse to process certificates when they recorded **Unlocked** in their UnlockDb.

Next, we show that validators that might process on the consensus path both a Cert (through checkpointing) and UnlockCert will arrive at the same execution result. We prove the case where an UnlockCert is ordered first. For this, we need to enhance the protocol of checkpointing in Sui to check the value of UnlockDb[ObjectKey] and ignore a Cert that tries to process a **Confirmed** ObjectKey, which is a straightforward change.

Theorem 6. *If a correct validator executes an UnlockCert certificate over ObjectKey as sequenced by the SMR engine, no correct validator will subsequently execute a conflicting Cert as sequenced by the SMR engine.*

Proof. The proof directly follows from the safety property of the SMR engine that all validators will process certificates in the same order. Hence, upon processing UnlockCert, all honest validators mark the execution of ObjectKey as confirmed by setting UnlockDb[ObjectKey] $\leftarrow$ **Confirmed** (Alg. 4 of Alg. 4). Then, Check (4.1) of Alg. 4 (and its dual added at the checkpoint algorithm) ensures that if any further Cert or UnlockCert with a conflict is given as input to the execution engine it is rejected. □

The converse can be proven in the same manner since we enhance the execution of Cert during the checkpoint process with updating UnlockDb[ObjectKey] $\leftarrow$ **Confirmed** after processing. Then all UnlockCert on the ObjectKey will be rejected at the Check (4.1) of Alg. 4.

Based on the above theorems, we can prove safety of the overall Stingray system that uses FastUnlock.

Theorem 7. *Stingray satisfies safety (Def. 3).*

Proof. Validity holds because validators execute only transactions with valid certificates.

Next, we prove global safety. For any given object O, let $T^O_p(t)$ be the set of transactions executed on that object by validator p up to time t. Due to Thm. 6, for any two validators p, q, $T^O_p(t) \subseteq T^O_q(t)$ or $T^O_q(t) \subseteq T^O_p(t)$. Thus, $\bigcup_{p \text{ honest}} T^O_p(t)$ is the set executed by some honest validator. Consider the sequence T^O made by arranging

transactions in this set in the order of the object version. Then, T^O contains all transactions in $\bigcup_{p \text{ honest}} T_p^O(t)$ and this sequence respects the application's validity constraint since an honest validator executed transactions in this sequence. Finally, let T be the merged sequence of T^O for all objects, where the merge preserves the partial order for each object. Due to the independence of different objects, T also satisfies the validity predicate, thus proving global safety. □

Liveness argument. Intuitively, we argue that FastUnlock—and its composition with normal fast path operations—neither deadlocks nor enables unjustified aborts (which could starve an object from progress).

Lemma 4 (Unlock Certificate Availability). *A correct user can obtain an unlock certificate UnlockCert over a valid ObjectKey.*

Proof. A correct validator always signs UnlockVote if it passes the check of Alg. 3. Well-formed UnlockRqt always come with a valid authentication path (Check (3.1)), and Check (3.2) always returns an UnlockVote. As a result, if UnlockRqt is disseminated to $2f + 1$ correct validators by a correct user, they will eventually all return an UnlockVote. The user then aggregates those votes into a unlock certificate UnlockCert over ObjectKey. □

Theorem 8 (FastUnlock Liveness). *If a correct and authorized user initiates a fast-unlock protocol, the ObjectKey in question will eventually admit a new transaction.*

Proof. A correct and authorized user will eventually generate an unlock certificate by Lem. 4. Additionally from the liveness property of SMR the unlock certificate will either eventually be added as part of the SMR output or the epoch will end. If the first happens by agreement of consensus the UnlockCert will be executed by all validators, leading to the termination of the fast-unlock protocol and an updated ObjectKey. If the epoch ends, all locks are dropped and liveness of all ObjectKey are automatically available for processing. □

Thm. 8 is sufficient for correct users as either they will manage to no-op an incorrect invocation of ObjectKey, drive the tranasction of a correct Tx to completion, or the epoch end will automatically unblock them. This means that there will always be an available ObjectKey to be modified.

Now that we proved that an authorized user will succeed into unblocking the ObjectKey we also need to show that an unauthorized user will not succeed into starving legitimate users from progress through abusing fast-unlock.

Theorem 9 (Starvation Freedom). *No user can successfully initiate a fast-unlock on an ObjectKey for which it cannot produce an Auth.*

Proof. All honest validators check the authorization vector Auth of the requesting user (Alg. 3 in Alg. 3). This means that no honest party will lock an object without an authorization, including slow parties that have not yet seen the ObjectKey which will reject or cache the request for later processing. As a result, by the model, there will never be sufficient UnlockVote to generate an UnlockCert driven by an unauthorized user. □

Theorem 10. *Stingray satisfies liveness (Def. 4).*

Proof. First we prove progress (Def. 4). Every transaction with a valid certificate will be eventually executed unless there is an UnlockCert containing Cert = **None**. Moreover, if the owners of the transaction's input objects do not equivocate, there will be no UnlockRqt for those object (recall that only the object owners can issue an UnlockRqt). This ensures progress.

Next, we prove eventual consistency (Def. 4). If a validator p_1 executes a transaction, it must have seen the transaction finalized, i.e., $2f + 1$ validators signed a certificate for that transaction. Therefore, at least $f + 1$ honest validators must have seen a certificate Cert for that transaction. If there is no UnlockCert for the transaction's input objects, then eventually, all honest validators will receive $2f + 1$ signatures on the certificate, and thereafter execute the transaction. If there is an UnlockCert for one of the transaction's input objects, then UnlockCert must contain Cert because at least one honest validator whose signature is in UnlockCert must have seen Cert. Therefore, even in this case, all honest validators will eventually execute the transaction. □

Generalization to multi-object unlock. The multi-object unlock protocol can be seen as a composition of many single-object unlock protocols (one per object) as well as a single commit protocol (for the accompanied transaction). As a result, the safety of the protocol follows from the fact that objects are independent of each other so if at least one has a prior certificate then the commit flow will lead to committing that prior certificate (which iteratively applies to all objects with prior certificates). If on the other hand, no object has a prior certificate then the workflow is the combination of the simple FastUnlock per object together with the shared-object path of committing the transactions of Sui which is safe as proven in the original Sui paper [12]. Second, we explore liveness. There are two cases: (1) all objects can be unlocked, (2) one or more objects are already certified. The first case is exactly the same as the simple protocol of Sec. 4.2 and a proof would follow exactly the same structure. For the second case, we first look into the base case of a single object that is already certified which is already proven in the previous sections. For more than one objects we can see that since the validator adds all certificates in their reply and then processes each certificate separately when handling the unlock cert then there is no interaction between certificate processing and can be considered a batch of independent requests.

Finally, for liveness the accompanied transaction might need to acquire locks. This is also an independent invocation of the Sui fast-path. As a result if the transaction is valid it will either succeed or blocks. In the latter case, the user will have to invoke fast-unlock again including in the set of to-unlock objects the newly blocked objects of the transaction. Given that there is a finite number of objects a user holds an unlock request will eventually succeed.

C Detailed Experimental Setup

This section complements Sec. 6.1 by specifying the network setup and machine specs used in the benchmarks presented in Sec. 6.

We deploy all systems on AWS, using `m5d.8xlarge` instances across 13 different AWS regions: N. Virginia (us-east-1), Oregon (us-west-2), Canada (ca-central-1), Frankfurt (eu-central-1), Ireland

(eu-west-1), London (eu-west-2), Paris (eu-west-3), Stockholm (eu-north-1), Mumbai (ap-south-1), Singapore (ap-southeast-1), Sydney (ap-southeast-2), Tokyo (ap-northeast-1), and Seoul (ap-northeast-2). Validators are uniformly distributed across those regions. Each machine provides 10 Gbps of bandwidth, 32 virtual CPUs (16 physical cores) on a 3.1 GHz Intel Xeon Skylake 8175M, 128 GB memory, and runs Linux Ubuntu server 22.04. We select these machines because they provide decent performance, are in the price range of "commodity servers", and satisfy the minimum required to run a Sui node as recommended by the Sui Foundation[7].

[7] https://docs.sui.io/guides/operator/validator-config